

plan

DS-API module to manage Plans and their contents in Pulse.

Omnichannel Plans

At the moment Omnichannel integration is still in beta. Feature dependencies are unstable for Omnichannel plans.

AVAILABLE_FILE_FORMATS module-attribute

```
AVAILABLE_FILE_FORMATS = ['csv', 'parquet']
```

DIFF_NAME_PLACEHOLDER module-attribute

```
DIFF_NAME_PLACEHOLDER = '{plan_name}_diff_{other_plan_name}'
```

DeployablePlanConfig module-attribute

```
DeployablePlanConfig = autoclass(MODEL_CLASSPATH + 'datascience.plan.DeployablePlanConfig')
```

FilterFeature module-attribute

```
FilterFeature = autoclass(DS_API_CLASSPATH + 'plan.features.FilterFeature')
```

FilterOperatorConfig module-attribute

```
FilterOperatorConfig = autoclass(MODEL_CLASSPATH + 'datascience.plan.operator.FilterOperatorConfig')
```

JChar module-attribute

```
JChar = autoclass('java.lang.Character')
```

LOGGER module-attribute

```
LOGGER = getLogger(__name__)
```

Plan

```
Plan(pyjnius_java)
```

Bases: [JavaWrapper](#)

Representing a Plan in Pulse.

CSV_SCHEMA class-attribute instance-attribute

```
CSV_SCHEMA = Enum('CSV_SCHEMA', _headers)
```

main_channel property [¶](#)

```
main_channel: Channel
```

secondary_channels property [¶](#)

```
secondary_channels: list[Channel]
```

subclasses class-attribute instance-attribute [¶](#)

```
subclasses: List[JavaWrapper] = []
```

add_empty_filter_operator [¶](#)

```
add_empty_filter_operator(name: str) -> FilterOperator
```

Adds a new filter operator to the Plan.

Parameters:

Name	Type	Description	Default
name ¶	str	The name of the new operator	<i>required</i>

Returns:

Type	Description
FilterOperator	The new operator

add_empty_list_operator [¶](#)

```
add_empty_list_operator(name: str) -> ListOperator
```

Adds a new list operator to the Plan.

Parameters:

Name	Type	Description	Default
name ¶	str	The name of the new operator.	<i>required</i>

Returns:

Type	Description
ListOperator	The new operator.

add_empty_map_operator [¶](#)

```
add_empty_map_operator(name: str) -> MapOperator
```

Adds a new map operator to the Plan.

Parameters:

Name	Type	Description	Default
<code>name</code> ¶	<code>str</code>	The name of the new operator.	<i>required</i>

Returns:

Type	Description
MapOperator	The new operator.

`add_empty_metric_operator` [¶](#)

```
add_empty_metric_operator(name: str) -> MetricOperator
```

Adds a new metric operator to the Plan.

Parameters:

Name	Type	Description	Default
<code>name</code> ¶	<code>str</code>	The name of the new operator.	<i>required</i>

Returns:

Type	Description
MetricOperator	The new operator.

`add_operator` [¶](#)

```
add_operator(operator: BaseOperator) -> BaseOperator
```

Adds a constructed operator to the Plan.

Parameters:

Name	Type	Description	Default
<code>operator</code> ¶	BaseOperator	Operator object.	<i>required</i>

Returns:

Type	Description
BaseOperator	The new operator.

`auto_rename_metrics` [¶](#)

```
auto_rename_metrics()
```

Automatically renames all the metrics based on their configuration.

Default naming: `lowercase({groupby_fields}_{aggregation}_{aggregation fields}_{window size}_{is_real_time})` eg. `accountid_avg_amount_1h_rt`. Advanced aggregations will be named: `lowercase({groupby_fields}_agg_`

`{window_size}_{is_real_time}`) . If a metric name is duplicated anywhere in the plan it appends "_n". Updates the references to the old names across the plan.

connect_operators [¶](#)

```
connect_operators(connection: Connection)
```

Adds a new connection between two Operators.

Parameters:

Name	Type	Description	Default
<code>connection</code> ¶	<code>Connection</code>	Connection between two operators.	<i>required</i>

connect_operators_names [¶](#)

```
connect_operators_names(op_names: list[str]) -> None
```

Connects the operators in the same order as they are indexed.

Only supports deployable linear plans.

Parameters:

Name	Type	Description	Default
<code>op_names</code> ¶	<code>list</code>	List of ordered operator names.	<i>required</i>

Raises:

Type	Description
<code>ValueError</code>	If the plan contains duplicated operator names. If there are not at least two operator names.
<code>TypeError</code>	If the operator name does not exist.

create_empty_deployable_plan `classmethod` [¶](#)

```
create_empty_deployable_plan(app: App, schema: Schema, timestamp_field_desc: str, name: str) -> Plan
```

Creates a new (empty) Deployable Plan in Pulse.

Parameters:

Name	Type	Description	Default
<code>app</code> ¶	<code>App</code>	Application <code>ds_api</code> object.	<i>required</i>
<code>schema</code> ¶	<code>Schema</code>	Pulse Schema	<i>required</i>
<code>timestamp_field_desc</code> ¶	<code>str</code>	Desc name of the schema timestamp field.	<i>required</i>

Name	Type	Description	Default
<code>name</code> ¶	<code>str</code>	Name of the new plan.	<i>required</i>

Returns:

Type	Description
<code>Plan</code>	New Plan

`create_empty_omnichannel_plan` [¶](#) `classmethod` [¶](#)

```
create_empty_omnichannel_plan(app: App, name: str, main_channel: Channel, secondary_channels: list[Channel] | None = None) -> Plan
```

Create a new empty Deployable Omnichannel Plan in Pulse.

Parameters:

Name	Type	Description	Default
<code>app</code> ¶	<code>App</code>	Pulse application ds_api object.	<i>required</i>
<code>name</code> ¶	<code>str</code>	Name of the new plan.	<i>required</i>
<code>main_channel</code> ¶	<code>Channel</code>	Main channel configuration.	<i>required</i>
<code>secondary_channels</code> ¶	<code>list of Channel</code>	Secondary channels. When not configured or if the list is empty, this is equivalent to create an empty deployable plan.	<code>None</code>

Returns:

Type	Description
<code>Plan</code>	New Plan.

`create_snapshot` [¶](#)

```
create_snapshot(schema_name: str | None = None, snapshot_name: str | None = None) -> JavaClass
```

Creates a Plan snapshot.

Parameters:

Name	Type	Description	Default
<code>snapshot_name</code> ¶	<code>str</code>	Name of the snapshot, by default "planname_yyyyMMdd_HHmm_ms".	<code>None</code>
<code>schema_name</code> ¶	<code>str</code>	If specified it exports the schema with this name, by default None.	<code>None</code>

Returns:

Type	Description
PlanSnapshot	PlanSnapshot JavaClass

diff [↗](#)

```
diff(other: Plan, show: bool = True, only_diff: bool = True, save: bool = False, save_as: Optional[str] = None) -> Optional[Plan]
```

Compare 2 plans. Differences can be displayed graphically when running inside a Jupyter Notebook.

Note

When creating a new plan from the diff, the schema and timestamp fields must be the in both plans. Also note that a renamed operator acts like a new operator for the diff, which means an operator that is not present in the 'other' plan will be discarded.

Danger

List and Sample operators are not supported.

Parameters:

Name	Type	Description	Default
other ↗	Plan	Plan to compare against. The 'new' plan. When pushing the diff to Pulse, this is the same as doing <code>other - self</code> .	<i>required</i>
show ↗	bool	Whether to display the differences as an HTML output. Only possible within a Jupyter Notebook. Defaults to True.	True
only_diff ↗	bool	If True only shows features with differences.	True
save ↗	bool	If True it outputs a new valid plan with the differences. This will override 'show'. Defaults to False.	False
save_as ↗	str	The name of the diff plan, if None it will be <code>"{plan_a}_diff_{plan_b}"</code> .	None

Raises:

Type	Description
NotImplementedError	When trying to show a diff outside a Jupyter Notebook.
ValueError	When the plans do not have the same schema or a different timestamp field. This is only raised when 'return_diff_plan' is set to true.

disconnect_operators [↗](#)

```
disconnect_operators(connection: Connection)
```

Deletes the connection between two operators.

Parameters:

Name	Type	Description	Default
<code>connection</code> ¶	Connection	Connection between two operators	<i>required</i>

`duplicate` [¶](#)

```
duplicate(new_name: Optional[str] = None, app: Optional[App] = None) -> Plan
```

Duplicates current plan to Pulse.

Optionally allows to duplicate the plan to a different app. This assumes the necessary dependencies are available, ie. Plan schema and lists referenced by a ListOperator.

Parameters:

Name	Type	Description	Default
<code>new_name</code> ¶	<code>str</code>	Desc name of the duplicated plan, by default will be the current name followed by '_duplicate'.	None
<code>app</code> ¶	App	Application ds_api object where the Plan should be created in, by default will be the current app.	None

Returns:

Type	Description
Plan	Duplicated plan.

Raises:

Type	Description
<code>ValueError</code>	If a plan with the same name as the duplicate already exists.

`from_csv` classmethod [¶](#)

```
from_csv(app: App, schema: Schema | None = None, timestamp_field_desc: str | None = None, name: str | None = None, path: Path | str | None = None, *, main_channel: Channel | None = None, secondary_channels: list[Channel] | None = None) -> Plan
```

Returns a new Deployable Plan with the configuration from the csv file.

The new Plan configuration will be updated in Pulse.

Parameters:

Name	Type	Description	Default
<code>app</code> ¶	App	Application ds_api object.	<i>required</i>
<code>schema</code> ¶	Schema	Pulse schema	None
<code>timestamp_field_desc</code> ¶	<code>str</code>	Desc name of the schema timestamp field.	None

Name	Type	Description	Default
<code>name</code> ↗	<code>str</code>	Name of the new plan	None
<code>path</code> ↗	<code>str</code> or <code>Path</code>	Path to where the csv will be read from.	None
<code>main_channel</code> ↗	Channel	If a main_channel is given, the schema, and timestamp_field_desc parameters are ignored.	None
<code>secondary_channels</code> ↗	list of <code>Channel</code>	Optionally pass the omnichannel configuration for the plan. Note that it is assumed the "schema" and "timestamp_field_desc" are properties of the main channel.	None

Notes

This method is intended to edit an existing Plan configuration previously exported.

General guidelines to edit the csv:

- The row order of the 'operator_name' will determine their position in the Pulse Plan.
- Pulse fields must be enclosed within the pattern `${field}`.
- Boolean values: {True, "TRUE", "True", "true", "T", "t", "1", 1} are accepted.
- Any feature to be removed should be on a single row and the field 'map_remove_fields' should be True.
- Feature updates will occur if a feature name is repeated.

Fields input format and considerations:

- A more extensive documentation on each of the fields can be found on the Feature class docstrings.
- operator_type: possible values are: map, metric, list, filter.
- expression: to be used for Filter, Map and List operators.
- metric_groupby_fields: formatted as a list of strings, eg. `['MerchantName', 'City']`.
- metric_aggregation: Pulse fields must be enclosed within the pattern `'${field_desc}'`. eg. `'avg(${amount})'`, `'countDistinct(${MerchantCity})'`, `'count()'`.
- metric_window_size: usual types "second", "minute", "hour", "day", "week". eg. 1 day.
- metric_is_real_time: boolean.
- metric_filter: Fields must be enclosed with the pattern `'${field}'` eg. `'${TransactionAmount} > 100 and ${Fraud} != false'`.
- metric_delay: int.
- metric_is_tumbling: boolean.
- metric_filter_incomplete_instances: boolean, must be the same for the whole operator.
- metric_timestamp_field: must be the same for the whole operator.
- map_field_type: possible values are: numerical, string, datetime, categorical.
- map_field_config: contains additional field configuration for datetime fields, possible values: nanoseconds, microseconds, milliseconds, seconds, minutes, hours, days. Or categories for categorical fields, eg. `['true', 'false']`.
- map_remove_fields: boolean.
- list_operation: get, or contains.
- list_field: desc of the list field.

Returns:

Type	Description
Plan	New Plan

Raises:

Type	Description
Exception	If the Plan configuration is invalid.

from_pandas classmethod [¶](#)

```
from_pandas(app: App, schema: Schema | None = None, timestamp_field_desc: str | None = None,
name: str | None = None, pd_df: DataFrame | None = None, *, main_channel: Channel | None = None,
secondary_channels: list[Channel] | None = None) -> Plan
```

Returns a new Deployable Plan with the configuration from the pandas dataframe.

The new Plan configuration will be updated in Pulse.

Parameters:

Name	Type	Description	Default
<code>app</code> ¶	App	Application ds_api object.	<i>required</i>
<code>schema</code> ¶	Schema	Pulse schema	None
<code>timestamp_field_desc</code> ¶	str	Desc name of the schema timestamp field.	None
<code>name</code> ¶	str	Name of the new plan	None
<code>pd_df</code> ¶	DataFrame	Pandas dataframe with the plan features.	None
<code>main_channel</code> ¶	Channel	If a main_channel is given, the schema, and timestamp_field_desc parameters are ignored.	None
<code>secondary_channels</code> ¶	list of Channel	Optionally pass the omnichannel configuration for the plan. Note that it is assumed the "schema" and "timestamp_field_desc" are properties of the main channel.	None

See Also

from_csv : For guidelines to input csv data in the right format.

Returns:

Type	Description
Plan	New Plan

Raises:

Type	Description
(JavaException , ValueError , TypeError)	If the Plan configuration is invalid.

`from_plan_snapshot` classmethod [¶](#)

```
from_plan_snapshot(app: App, plan_snapshot: JavaWrapper) -> Plan
```

Returns a new Plan based on the snapshot.

Parameters:

Name	Type	Description	Default
<code>app</code> ¶	<code>App</code>		<i>required</i>
<code>plan_snapshot</code> ¶	<code>JavaWrapper</code>		<i>required</i>

Returns:

Type	Description
<code>Plan</code>	New Plan.

`get_feature_dependencies` [¶](#)

```
get_feature_dependencies(feature: BaseFeature, ascending=True) -> set[FeatureUID]
```

Gets a set of feature dependencies calculated recursively.

Feature dependencies are represented as a unique tuple of feature name and operator name. Considers the filter operator as a feature with dependencies.

Parameters:

Name	Type	Description	Default
<code>feature</code> ¶	<code>BaseFeature</code>	Feature to get the dependencies from.	<i>required</i>
<code>ascending</code> ¶	<code>bool</code>	If True it will look for dependencies in upstream operators, including the direct dependencies, otherwise it will look downstream. (Default = True).	<code>True</code>

Returns:

Type	Description
<code>set</code>	Sequence of <code>FeatureUID</code> with the names of the feature dependencies calculated recursively and the operator name where they are defined.

`get_feature_operators` [¶](#)

```
get_feature_operators() -> List[BaseFeaturesOperator]
```

Return this plan's 'feature operators', i.e., map, list, and metric operators. Filter operators are excluded.

Returns:

Type	Description
<code>List of BaseFeaturesOperator</code>	

Type	Description
	List of feature operators in the plan.

`get_features` [↗](#)

```
get_features() -> List[BaseFeature]
```

Return all features in the plan excluding "schema features".

`get_features_dependencies_by_type` [↗](#)

```
get_features_dependencies_by_type() -> Dict[str, Dict[str, Set[str]]]
```

Gets a set of feature dependencies calculated recursively.

Feature dependencies are represented as a unique tuple of feature name and operator name.

Considers the filter operator as a feature with dependencies.

If we have the following graph of dependencies for features A, B and C and specific dependency types:

```
[C]--(map)-->[B]--(aggregation)-->[A]
```

Feature C is considered an "aggregation" dependency of Feature A (regardless of its dependency type to B) because it is a dependency of the intermediate Feature B that is used in the aggregation expression of Feature A.

Returns:

Type	Description
<code>Dict[str, Dict[str, Set[str]]]</code>	Dictionary of features' dependencies by type. The keys are the names of the features and the values are the dicts of upstream dependencies by type.

`get_features_dependencies_graph` [↗](#)

```
get_features_dependencies_graph() -> DiGraph
```

Gets a directed graph object from NetworkX built with the dependencies between the Plan features.

Nodes are unique tuples of feature and operator names.

Considers the filter operator as a feature with dependencies.

Returns:

Type	Description
<code>DiGraph</code>	DiGraph

`get_operator_by_name` [↗](#)

```
get_operator_by_name(name: str) -> Optional[BaseOperator]
```

Selects the Pulse Operators with the matching name.

If more than one operator is found then the first one is returned.

Parameters:

Name	Type	Description	Default
<code>name</code> ¶	<code>str</code>	The name of the operator.	<i>required</i>

Returns:

Type	Description
<code>Optional[BaseOperator]</code>	The Pulse operator if matching name or None.

`get_operator_connections` [¶](#)

```
get_operator_connections() -> List[Connection]
```

Gets the operator connections.

Returns:

Type	Description
<code>list of Connections</code>	list of Connections

`get_operators` [¶](#)

```
get_operators() -> List[BaseOperator]
```

Gets the Plan operators.

Returns:

Type	Description
<code>list of BaseOperator</code>	List of operators in the plan.

`get_type` [¶](#)

```
get_type() -> str
```

Returns plan type.

Returns:

Type	Description
<code>str</code>	Either 'DEPLOYABLE' or 'NON_DEPLOYABLE'

`remove_feature_and_dependants` [¶](#)

```
remove_feature_and_dependants(feature: BaseFeature)
```

Removes the given feature and all the downstream dependencies.

Parameters:

Name	Type	Description	Default
	<code>BaseFeature</code>	Feature to be deleted and from which to remove downstream dependencies.	<i>required</i>

Name	Type	Description	Default
feature ↗			

remove_operator [↗](#)

```
remove_operator(operator: BaseOperator)
```

Removes an operator from the Plan.

Automatically deletes any connections involving this operator.

Parameters:

Name	Type	Description	Default
operator ↗	BaseOperator	Operator to be removed.	<i>required</i>

rename_feature [↗](#)

```
rename_feature(old_name: str, new_name: str)
```

Renames the given features updating the references to the old names across the plan.

Parameters:

Name	Type	Description	Default
old_name ↗	str	Old name to change from.	<i>required</i>
new_name ↗	str	New name to change to.	<i>required</i>

run [↗](#)

```
run(input_datasources: list[DataSource], output_names: list[str], output_paths: list[str], *, secondary_datasources: dict[str, DataSource] | None = None, job_profile: JobProfile | None = None, job_properties: dict[str, str] | None = None, wait: bool = False, file_format: str = 'parquet', partitioning_config: PartitioningConfig = AdaptivePartitioningConfig(), **kwargs: bool | str | set[str]) -> str
```

Runs this Plan, supports deployable and non-deployable plans.

The plan will be run with the Pulse default config in place unless specified another Pulse job config and/or spark parameters.

Parameters:

Name	Type	Description	Default
input_datasources ↗	list	List of DataSource objects.	<i>required</i>
output_names ↗	list	Desc names of the output data sources.	<i>required</i>
	list	Paths to the output datasets.	<i>required</i>

Name	Type	Description	Default
<code>output_paths</code> ↗			
<code>secondary_data_sources</code> ↗	<code>dict[str, DataSource]</code>	Mapping of channel name to input Data source for secondary channels. For example <code>{"cards": <DataSource: CardsClean>}</code> .	None
<code>job_profile</code> ↗	JobProfile	Pulse UI custom job profile.	None
<code>job_properties</code> ↗	<code>dict</code>	Additional spark properties, eg. <code>{"spark.executor.memory": "1g"}</code> , by default no additional spark properties are added.	None
<code>wait</code> ↗	<code>bool</code>	Whether to wait for the plan execution to finish. The plan execution outcome will be logged. By default, False.	False
<code>file_format</code> ↗	<code>str</code>	Chosen file format for outputs. By default, parquet.	'parquet'
<code>partitioning_config</code> ↗	PartitioningConfig	The configuration for the partitioning approach to use when executing the plan. By default, <code>AdaptivePartitioningConfig()</code> .	AdaptivePartitioningConfig()

Other Parameters:

Name	Type	Description
<code>delimiter</code>	<code>str</code>	Delimiter character for the output datasets, by default ",". Only used if output format is csv.
<code>line_sep</code>	<code>str</code>	Line separator for the output datasets, by default "\n". Only used if output format is csv.
<code>quote_char</code>	<code>str</code>	Quotation character for the output datasets, by default "". Only used if output format is csv.
<code>header</code>	<code>bool</code>	Write out the column names, by default True. Only used if output format is csv.
<code>missing_values</code>	<code>set of str</code>	Missing values for the output datasets, by default "", empty string. Only used if output format is csv.

Returns:

Type	Description
<code>str</code>	Name of the plan execution.

`select_features` [↗](#)

```
select_features(feature_names: List[str])
```

Removes from the Plan any feature that is not needed to output the given list of features.

If a feature is updated, thus having duplicated names, it will conservatively keep all upstream dependencies.

Parameters:

Name	Type	Description	Default
<code>feature_names</code> ¶	<code>list of str</code>	List of feature names to keep.	<i>required</i>

`to_csv` [¶](#)

```
to_csv(path: Union[Path, str])
```

Exports the deployable Plan to a tabular csv format.

Parameters:

Name	Type	Description	Default
<code>path</code> ¶	<code>Path or str</code>	Path to where the csv will be saved.	<i>required</i>

`to_pandas` [¶](#)

```
to_pandas() -> DataFrame
```

Exports the deployable Plan to a pandas dataframe.

Returns:

Type	Description
<code>DataFrame</code>	Pandas dataframe containing features and all their editable attributes.

`to_pulse` [¶](#)

```
to_pulse()
```

Sends the current plan configuration of Operators and Features to Pulse.

Raises:

Type	Description
<code>JavaException</code>	If the Plan configuration is not valid.

`update_from_csv` [¶](#)

```
update_from_csv(path: Union[Path, str])
```

Updates this Deployable Plan with the configuration from the csv file.

Parameters:

Name	Type	Description	Default
<code>path</code> ¶	<code>str or path</code>	Path to where the csv will be read from.	<i>required</i>

See Also

from_csv : For guidelines to input csv data in the right format.

Raises:

Type	Description
(<code>JavaException</code> , <code>ValueError</code> , <code>TypeError</code>)	If the Plan configuration is invalid.

update_from_pandas [↗](#)

```
update_from_pandas(pd_df: DataFrame)
```

Updates this Deployable Plan with the configuration from the pandas dataframe.

Parameters:

Name	Type	Description	Default
<code>pd_df</code> ↗	<code>DataFrame</code>	Pandas dataframe with the plan features.	<i>required</i>

See Also

from_csv : For guidelines to input csv data in the right format.

Raises:

Type	Description
(<code>JavaException</code> , <code>ValueError</code> , <code>TypeError</code>)	If the Plan configuration is invalid.

validate [↗](#)

```
validate()
```

Validates the current state of the Plan including any alterations to Operators and Features.

Raises:

Type	Description
<code>JavaException</code>	If the Plan configuration is invalid.